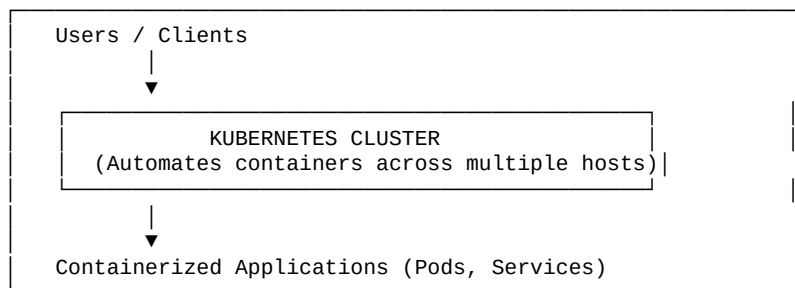


KUBERNETES ARCHITECTURE & CORE CONCEPTS

1. What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. Originally developed by Google, it is now maintained by the Cloud Native Computing Foundation (CNCF).

Kubernetes Overview:



2. Why Do We Need Kubernetes?

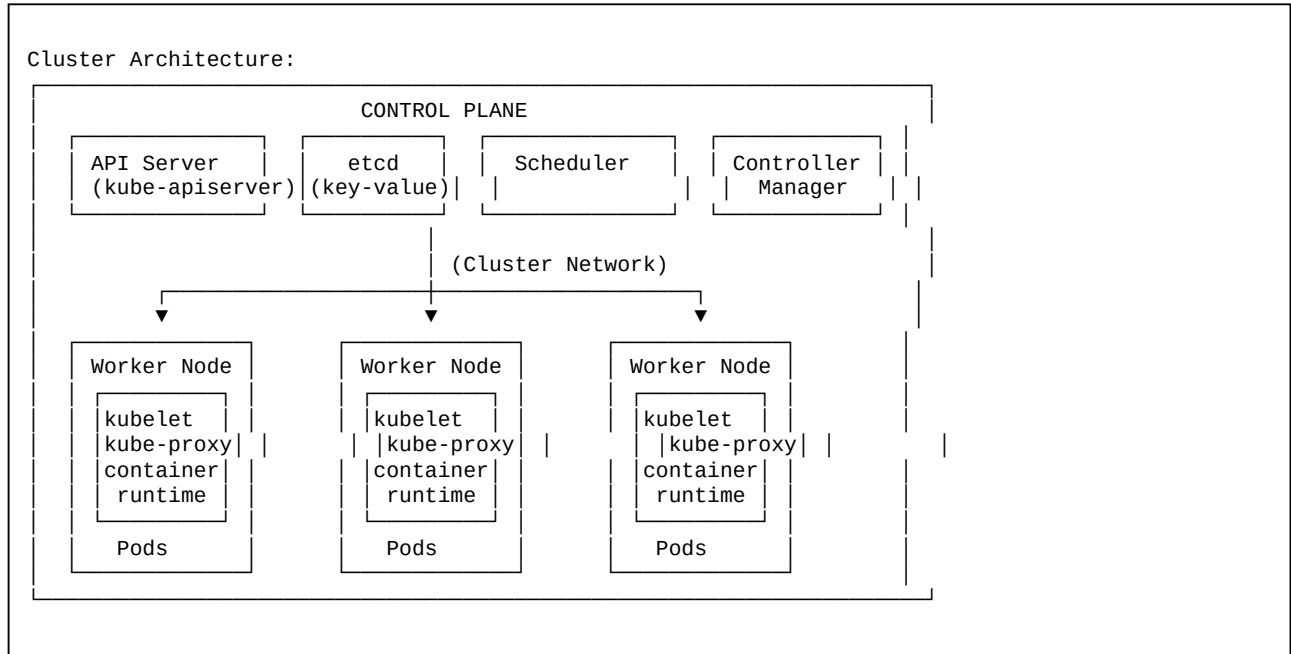
- **Manual Management is Hard:** Running containers at scale requires handling failures, updates, networking, storage, etc. Kubernetes automates these tasks.
- **High Availability:** Automatically restarts failed containers, reschedules them on healthy nodes, and ensures desired state is maintained.
- **Scaling:** Scale applications up/down based on CPU/memory usage or custom metrics (Horizontal Pod Autoscaler).
- **Zero-downtime Deployments:** Rolling updates, canary deployments, blue-green deployments without service interruption.
- **Resource Efficiency:** Schedules containers based on resource requests/limits, improving infrastructure utilization.
- **Service Discovery & Load Balancing:** Automatically assigns DNS names to services and distributes traffic across healthy pods.
- **Storage Orchestration:** Mounts persistent storage (local, cloud, NFS) to containers as needed.
- **Self-healing:** Reschedules failed containers, replaces unhealthy pods, and kills unresponsive containers based on liveness/readiness probes.
- **Secret & Configuration Management:** Securely stores and injects secrets and config data without rebuilding images.

3. Key Features of Kubernetes

Feature	Description
Automated Scheduling	Assigns containers to nodes based on resource needs, constraints, and affinity rules. Helps optimize resource usage and availability.
Self-Healing	Automatically restarts failed containers, replaces and reschedules containers when nodes die, kills containers that don't respond to user-defined health checks.
Horizontal Scaling	Scale applications manually or automatically (HPA) based on CPU/memory usage or custom metrics. Can also scale replicas of pods.
Rolling Updates & Rollbacks	Gradually updates pods to new version with zero downtime. If something goes wrong, automatically rollback to previous revision.
Service Discovery & Load Balancing	Assigns stable IP addresses and DNS names to services, and load-balances traffic across healthy pods.
Storage Orchestration	Automatically mount storage systems (local, cloud, NFS, etc.) to containers with Persistent Volumes (PV).
Batch Execution (Jobs)	Manages batch and CI/CD workloads, replacing failed containers or running to completion.
Config and Secret Management	Separates configuration and sensitive data (secrets) from application images, making images reusable and secure.
Multi-Environment Consistency	Same Kubernetes manifests work across dev, test, staging, production, and different cloud providers.

4. Kubernetes Architecture Overview

Kubernetes follows a master-worker architecture. The cluster consists of a Control Plane (Master) and multiple Worker Nodes.



5. Control Plane Components (Master Node)

5.1 API Server (kube-apiserver)

Central management hub. All administrative operations (kubectl, clients) interact only with API Server. It validates and processes REST requests, updates etcd.

5.2 etcd

Distributed key-value store. Stores the entire cluster state: configurations, secrets, deployments, pods, services. Consistent and highly available. Backup is critical.

5.3 Scheduler (kube-scheduler)

Assigns unscheduled Pods to appropriate Worker Nodes based on resource requirements, affinity/anti-affinity rules, and constraints.

```
# Scheduler decisions consider:
- Resource requests/limits (CPU, memory)
- Node selectors / affinity
- Taints and tolerations
- Inter-pod anti-affinity
```

5.4 Controller Manager (kube-controller-manager)

Runs controller loops: Node Controller, Replication Controller, Endpoint Controller, Service Account Controller, etc. Watches desired state via API Server and makes changes to reach desired state.

6. Worker Node Components

6.1 Kubelet

Primary node agent. Registers node with cluster, watches PodSpec (via API Server), ensures containers are healthy, reports node/pod status to control plane.

6.2 Kube-proxy

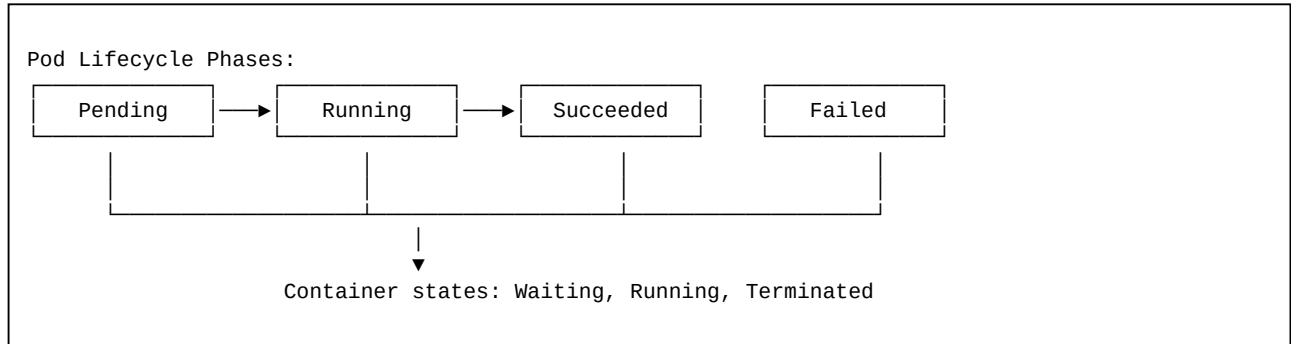
Network proxy and load balancer. Maintains network rules on each node (iptables/IPVS). Enables service abstraction (ClusterIP, NodePort, etc.).

6.3 Container Runtime

Software to run containers (containerd, CRI-O, Docker). Kubelet uses CRI (Container Runtime Interface) to communicate with runtime.

7. Pods: Container Lifecycle and Multi-Container Pods

Pod is the smallest deployable unit in Kubernetes, consisting of one or more containers sharing network, storage, and namespace. Containers in a pod are co-located and co-scheduled.



7.1 Container Lifecycle (within Pod)

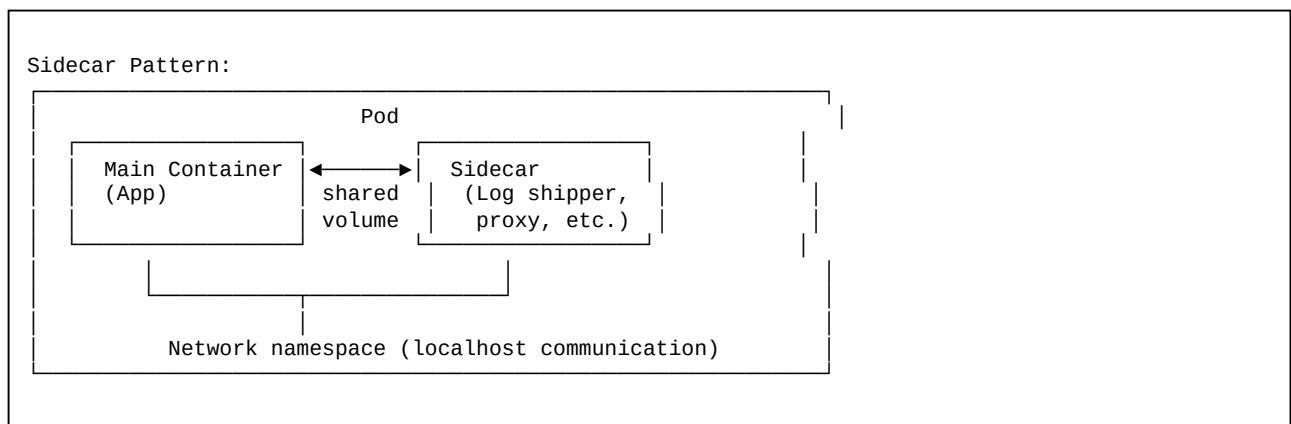
- **Waiting:** Container needs to pull image or waiting for dependencies.
- **Running:** Container executing normally.
- **Terminated:** Container exited (success/error) – may restart based on restartPolicy.

```
# Check pod status
kubectl get pods
kubectl describe pod mypod

# Container restart policy: Always, OnFailure, Never
```

7.2 Multi-Container Pods (Sidecar Pattern)

Sidecar containers extend/enhance main container functionality. Common scenarios: logging, monitoring, proxying, config reload.



```
# Example sidecar (log rotation + main nginx)
apiVersion: v1
kind: Pod
metadata:
  name: web-with-sidecar
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: logs
          mountPath: /var/log/nginx
    - name: logrotator
      image: alpine
      command: ["sh", "-c", "tail -F /var/log/nginx/access.log"]
      volumeMounts:
        - name: logs
          mountPath: /var/log/nginx
  volumes:
    - name: logs
      emptyDir: {}
```

7.3 Other Multi-container Patterns

- **Ambassador:** Proxy connections to external services (e.g., connect to Redis via ambassador).
- **Adapter:** Transform data output from main container into format required by external system.
- **Init Containers:** Run before main containers to setup prerequisites (database schema, wait for service).

8. Two Ways of Creating Pods: Imperative vs Declarative

In Kubernetes, you can create pods (and other resources) using two distinct approaches: imperative commands and declarative manifests.

8.1 Imperative Approach

Directly command the API server using kubectl commands. Best for quick testing, debugging, or one-off operations.

```
# Create a single pod imperatively
kubectl run my-nginx --image=nginx

# Create a pod with specific labels and ports
kubectl run my-app --image=myapp:1.0 --port=8080 --labels="env=dev,app=backend"

# Create a pod in a specific namespace
kubectl run busybox --image=busybox --namespace=mynamespace -- sleep 3600

# Expose a pod as a service imperatively
kubectl expose pod my-nginx --port=80 --type=ClusterIP

# Edit a pod directly (if allowed)
kubectl edit pod my-nginx

# Delete a pod imperatively
kubectl delete pod my-nginx
```

8.2 Declarative Approach

Define the desired state of resources in YAML/JSON manifests and apply them to the cluster. Best for production, automation, and version control.

```
# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-declarative
  labels:
    app: nginx
    environment: production
spec:
  containers:
  - name: nginx
    image: nginx:1.23
    ports:
    - containerPort: 80
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

```
# Apply the manifest (create/update)
kubectl apply -f pod.yaml

# Delete the resource using the same manifest
kubectl delete -f pod.yaml

# Apply all manifests in a directory
kubectl apply -f ./manifests/

# Get the pod status
kubectl get pod nginx-declarative
```

8.3 Imperative vs Declarative Comparison

Feature	Imperative	Declarative
Operation style	Commands (kubectl run, create, delete)	Apply YAML manifests (kubectl apply -f)
Learning curve	Low (familiar CLI style)	Medium (YAML syntax, schema)
Reproducibility	Low (steps not tracked)	High (manifests as code)
CI/CD friendly	Limited	Excellent (git versioning, diff)
Use case	Development, test, debugging	Production, automation, collaboration

Best Practice: Use imperative for local experimentation and one-off tasks; use declarative (YAML/JSON) for all production workloads and any object you need to track, version, or rollback.

9. Docker vs Kubernetes

Aspect	Docker	Kubernetes
Primary Role	Container runtime + image management	Container orchestration (scheduling, scaling, service discovery)
Scope	Single node (or Swarm for multi-node)	Multi-node cluster (auto-scaling, self-healing)
Auto-healing	Limited (restart policies)	Restarts, reschedules, replaces failed containers
Service Discovery	Manual via links or custom networks	Built-in via Services and Ingress
Scaling	Manual (docker-compose scale)	Automatic (HPA, replicas, cluster autoscaler)
Load Balancing	User-managed (reverse proxy)	Built-in (kube-proxy, Ingress controllers)
Configuration Management	Env files, Compose	ConfigMaps, Secrets, Helm
Ecosystem	Docker Compose, Swarm	Helm, Operators, Istio, Prometheus

Docker provides containerization; Kubernetes provides orchestration. They are complementary. Many Kubernetes clusters use containerd (or Docker via CRI). Modern Kubernetes no longer directly requires Docker as runtime but can use it via cri-dockerd.

10. Kubernetes Distributions

Distributions package Kubernetes with additional tools, installation, management and operational ease.

10.1 Popular Distributions

Distribution	Description	Use Case
kubeadm (upstream)	Official tool to bootstrap a minimal conformant cluster.	Learning, production clusters (DIY control plane)
minikube	Single-node cluster for local development.	Local testing, development
kind	Kubernetes in Docker – multi-node clusters inside Docker containers.	CI/CD, testing
k3s	Lightweight certified Kubernetes (by Rancher). Ideal for edge/IoT.	Resource constrained environments, edge
k0s	Zero-friction, single-binary Kubernetes distribution.	Simple, secure clusters
OpenShift	Enterprise Kubernetes from Red Hat with enhanced security and developer tools.	Enterprise, DevSecOps
EKS (AWS)	Managed Kubernetes on AWS.	Cloud native, AWS integration
AKS (Azure)	Managed Kubernetes on Microsoft Azure.	Cloud native, Azure integration
GKE (Google)	Managed Kubernetes on Google Cloud.	Cloud native, Google integration

10.2 Popularity Factors

- Managed distributions (EKS, AKS, GKE) reduce operational overhead.
- Lightweight (k3s, k0s) for edge and small deployments.

- Upstream (kubeadm) for full control and standardization.

11. kops and kubeadm

Tools Comparison:

kubeadm	kops
Initializing clusters on any machine (Linux)	Provisioning production-grade clusters on AWS (and other cloud providers) with HA, networking (VPC) and bastion

11.1 kubeadm

Official Kubernetes tool to set up a secure, best-practices cluster. Performs pre-flight checks, generates certificates, sets up control plane components, joins worker nodes.

```
# Initialize control plane
kubeadm init --pod-network-cidr=10.244.0.0/16

# Join worker node (command generated by init)
kubeadm join :6443 --token --discovery-token-ca-cert-hash sha256:

# Upgrade cluster
kubeadm upgrade plan
kubeadm upgrade apply v1.29
```

Use cases: On-premises, self-managed clouds, learning, custom infrastructure.

11.2 kops (Kubernetes Operations)

Automates provisioning and management of Kubernetes clusters on AWS (and other clouds via terraform). Handles infrastructure: VPC, EC2, load balancers, autoscaling groups, IAM roles.

```
# Install kops
curl -Lo kops https://github.com/kubernetes/kops/releases/download/$(curl -s https://api.github.com/repos/kubernetes/kops/releases/latest | grep tag_name | cut -d '"' -f 4)/kops-linux-amd64
chmod +x kops && sudo mv kops /usr/local/bin/

# Create cluster definition
export NAME=mycluster.k8s.local
export ZONES=us-east-1a,us-east-1b
kops create cluster --name=${NAME} --zones=${ZONES} --master-size=t3.medium --node-size=t3.medium

# Review and apply
kops update cluster --name ${NAME} --yes

# Validate
kops validate cluster --wait 10m

# Delete cluster
kops delete cluster --name ${NAME} --yes
```

Features: Highly Available, rolling updates, cluster add-ons, configuration stored in S3.

11.3 Comparison: kubernetes vs kops

Feature	kubernetes	kops
Environment	Any Linux (bare metal, VMs, cloud)	AWS (primary), Google Cloud, OpenStack, etc.
Infrastructure provisioning	Manual or external (Terraform, Ansible)	Automated (creates VPC, subnets, ASG)
Node management	Join command; separate tools for scaling	Integrated via instance groups, autoscaling
High Availability	Can set up multiple control planes manually	Built-in HA support (multiple masters)
State storage	Local files or external etcd	S3 bucket (cluster spec)
Complexity	Low-level, flexible	Higher abstraction, AWS specific

Summary:

- Use **kubernetes** for full control, on-prem, or when you already have underlying infrastructure.
- Use **kops** to quickly spin up production-grade AWS clusters with best practices.
- Use **minikube** or **kind** for local development.
- Use **k3s** for lightweight / edge environments.